

Code review — academy

Runde de review pe proiectul tău, cea mai nouă sus.

Proiect: mini-ERP de academie, C# consolă, fără framework. Regula fișierului: runda nouă se adaugă **sus**. Rundele vechi rămân dedesubt, condensate.

Runda 5 — 2026-09-15 — sha `eb5c7a8`

Scop: verificat commit-ul `eb5c7a8` „enrolments refactoring” — reacția ta la runda 4. Metodă: `dotnet build` pe codul tău, apoi o copie în scratchpad pe care am pornit efectiv repository-urile și ecranul studentului. Codul tău nu a fost modificat.

Poarta de compilare — TRECE

```
Build succeeded.  
    33 Warning(s)  
    0 Error(s)
```

Prima dată în nouă commit-uri. Nu e o formalitate: de aici încolo pot rula codul tău în loc să repar o copie ca să-l pot citi. Ține regula asta — e cea care schimbă tot restul.

Ce ai închis, și e partea grea a runde trecute

- **B1**  **ÎNCHIS.** `Enrolment : IEntity`, `EnrolmentRepository` cu constructor `: base(new EnrolmentTextMapper(), ...)`, `EnrolmentTextMapper` și `EnrolmentMapper` scrise, iar **toți** apelanții actualizați — `EnrolmentService`, `ViewStudent`. Cele 6 erori ascunse pe care ți le-am listat, toate rezolvate. E prima dată când ai urmărit un tipar până la capăt în loc să reperi o linie.
- **Enrolment.Id** e acum **private set**, cu constructor de rehidratare și un al doilea care generează `Guid`-ul prin `: this(...)`. E exact tiparul `Book.Id` pe care ți l-am dat în august, aplicat singur, fără să ți-l cer.
- **B7**  **ÎNCHIS.** Cele două liste paralele au dispărut. `EnrolmentService` primește repository-ul din afară, are o singură sursă de adevăr, iar `ReadEnrolments` / `EnrolmentsListToString` / `Save`-ul

propriu s-au dus. Serviciul a scăzut de la 146 la 79 de linii și face mai mult.

- **DeleteEnrolment** **verifică și aruncă** — n-a fost cerut, l-ai făcut pentru că frații lui fac la fel.
- **M3** ✓ **EnrolmentMapper** scris și **static** · **M7** ✓ **InscriereCurs** trece prin **CreateEnrolment**, nu mai atinge lista · **C2** ✓ cele 28 de linii comentate din **User.cs**, șterse.
- **B3** ⚠ **trei sferturi** — trei din patru **Int32.Parse** șterse.

Refactorul de **Enrolment** e, ca structură, corect. Dacă îl compar cu **Books**, e aceeași formă.

Ce a mers prost

Un singur lucru, dar l-ai făcut în trei locuri: **ai pus prefix de tip în mapper-e fără să pui prefixul și în fișiere**. Rezultatul, rulat:

```
=== se construiesc repository-urile? ===
UserRepository:      OK
BookRepository:      FormatException -> Unrecognized Guid format.
EnrolmentRepository: FormatException -> Unrecognized Guid format.
```

Books mergea de la prima rundă. Acum nu mai merge. Detalii la B1.

🔴 Critice

B1 — Ai stricat **Books**, singurul modul care a mers în toate cele patru runde

Books/Mappers/BookTextMapper.cs:10 · **:17** · **Data/books.txt** · **ViewStudent.cs:16**

CUM E ACUM: rulat pe copie, construind ecranul pe care îl vede un student după login:

```
=== ecranul studentului ===
FormatException -> Unrecognized Guid format.
at Guid.Parse(ReadOnlySpan`1 input)
at BookTextMapper.FromText(String text) in BookTextMapper.cs:line 17
```

ViewStudent crapă în **constructor** (**:16**, **new BookRepository()**), înainte să afișeze primul rând de meniu.

Ce ai schimbat:

```
// ToText
return "BOOK," + item.Id + "," + item.StudentId + ...

// FromText
return new Book(Guid.Parse(cuv[1]), Guid.Parse(cuv[2]),
               cuv[3], DateTime.Parse(cuv[4]));
```

Ce scrie în `Data/books.txt` :

```
366b1330-...,d383e75c-...,Cartea1,2025-12-03
```

Niciun `BOOK,` . Deci `cuv[1]` nu e `Id` , e `StudentId` ; `cuv[2]` nu e `StudentId` , e `"Cartea1"` ; `Guid.Parse("Cartea1")` → `FormatException` .

DE CE: merită înțeles de ce ai făcut-o, pentru că intuiția din spate e bună și doar locul e greșit.

Prefixul de tip există ca să răspundă la o întrebare: „**linia asta ce fel de obiect descrie?**” Întrebarea are sens numai când un fișier — și un repository — conține **mai multe tipuri** care trebuie deosebite între ele. `users.txt` ține studenți, profesori și admini laolaltă, deci acolo întrebarea e reală, iar prefixul e răspunsul: e fix miezul lui T3, Factory Method.

`books.txt` ține un singur tip. `enrolments.txt` ține un singur tip. Acolo nu există nicio întrebare de pus, deci prefixul nu rezolvă nimic — dar **costă**: schimbă formatul fișierului. Ai aplicat răspunsul într-un loc unde nu se pusese întrebarea.

Și dedesubt stă regula care se plătește toată viața: **un format are două capete**. Cititorul și fișierul sunt o înțelegere între două părți. Când muți o parte, cealaltă trebuie mutată în aceeași ședință, sau nu se mută niciuna. Aici ai mutat cititorul și ai lăsat fișierul. Runda trecută, la `users.txt` , ai făcut exact invers: ai mutat fișierul și ai lăsat cititorul. Aceeași greșeală, în oglindă.

Semnalul care ți-ar fi spus asta în 5 secunde: ai schimbat un mapper și **n-ai pornit aplicația**. `Books` e modulul pe care îl atingi cel mai des; e și singurul care mergea.

FIX: scoate prefixul din `BookTextMapper` , în ambele metode. `Book` e un singur tip; n-are ce deosebi.

```

public string ToText(Book item)
{
    return item.Id + "," + item.StudentId + "," + item.BookName
        + "," + item.CreatedAt.ToString("yyyy-MM-dd");
}

public Book FromText(string text)
{
    string[] cuv = text.Split(',');

    return new Book(Guid.Parse(cuv[0]), Guid.Parse(cuv[1]),
        cuv[2], DateTime.Parse(cuv[3]));
}

```

B2 — EnrolmentTextMapper nu se poate citi pe el însuși. Fișier nou-nouț, round-trip rupt din prima

Enrolments/Mappers/EnrolmentTextMapper.cs:10 · :17 · Data/enrolments.txt

CUM E ACUM: am luat o înscriere, am scris-o cu `ToText` și am dat textul înapoi lui `FromText`. Același obiect, același mapper, patru rânduri distanță:

```

ToText    -> 46a77c08-...,5bb30bbb-...,2f2ae25f-...,
           9/15/2026 6:01:35 PM
FromText  -> FormatException: Unrecognized Guid format.

```

Și, în consecință, `new EnrolmentRepository()` crapă la pornire pe prima linie din `enrolments.txt`.

Sunt două nepotriviri suprapuse:

```
// ToText -> 4 campuri, FARA prefix
return item.Id + "," + item.StudentId + "," + item.CourseId
    + "," + item.CreatedAt;

// FromText -> asteapta 5 campuri, CU prefix
return new Enrolment(Guid.Parse(cuv[1]), Guid.Parse(cuv[2]),
    Guid.Parse(cuv[3]), DateTime.Parse(cuv[4]));
```

DE CE: e B4 din runda 3, exact aceeași formă — și tocmai l-ai închis corect la

`Student` / `Teacher` / `Admin` . Ce s-a întâmplat e că ai închis **instanțele**, nu **regula**: în fișierul nou ai scris cele două jumătăți separat, iar ele s-au despărțit din nou, la fel de tăcut.

Proba pe care ți-am dat-o atunci e singura care prinde asta și e de trei rânduri: scrie un obiect, citește-l înapoi, compară. Dacă o rulai o dată pe `Enrolment` , nu mai ajungea în commit.

Mai e ceva, mai subtil, în `item.CreatedAt` fără format. `DateTime.ToString()` fără argument folosește **cultura curentă a mașinii**. La mine a ieșit `9/15/2026 6:01:35 PM` . Pe Windows-ul tău, cu setări românești, iese `15.09.2026 18:01:35` . Iar în culturi în care separatorul de dată sau de oră e virgula, șirul ăla intră în mijlocul unui CSV **separat prin virgulă** și rupe toate câmpurile de după el. Formatul pe disc nu trebuie să depindă niciodată de setările calculatorului pe care rulează: pentru asta poți explicit `ToString("yyyy-MM-dd")` , ca în `BookTextMapper` — pe care l-ai scris corect acum două luni.

FIX:

```
public string ToText(Enrolment item)
{
    return item.Id + "," + item.StudentId + "," + item.CourseId
        + "," + item.CreatedAt.ToString("yyyy-MM-dd");
}

public Enrolment FromText(string text)
{
    string[] cuv = text.Split(',');

    return new Enrolment(Guid.Parse(cuv[0]), Guid.Parse(cuv[1]),
        Guid.Parse(cuv[2]), DateTime.Parse(cuv[3]));
}
```

Apoi rulează proba, o dată, înainte de commit:

```
var m = new EnrolmentTextMapper();
var e = new Enrolment(Guid.NewGuid(), Guid.NewGuid(), DateTime.Now);
var e2 = m.FromText(m.ToText(e));
Console.WriteLine(e.Id == e2.Id && e.CourseId == e2.CourseId);
```

B3 — `Save()` pe useri scrie `USER,` peste tot. Distruge exact prefixele de tip pe care le-ai pus tu

`Users/Mappers/UserTextMapper.cs:10` · `Data/users.txt`

CUM E ACUM: `users.txt` înainte, cu tipurile puse de tine runda trecută:

```
STUDENT,d383e75c-...,Stefan,Scarlat,stefanel.scarlat@gmail.com,18
ADMIN,9c99cd02-...,Alex,Grozavu,gicu@hotmail.com,48,10000,gicuEgrozav
```

Am construit `UserRepository` și am chemat `Save()`. După:

```
USER,d383e75c-...,Stefan,Scarlat,stefanel.scarlat@gmail.com,18
USER,7b78b29c-...,Alex,Bordei,alex.bordei@yahoo.com,27
...
USER,9c99cd02-...,Alex,Grozavu,gicu@hotmail.com,48
```

Adminul a devenit `USER` și și-a pierdut `10000` și `gicuEgrozav`.

DE CE: runda trecută `Save()` ștergea prefixul. Acum scrie unul — dar mereu același, `"USER,"`, pentru că `ToText(User item)` se uită la lume prin fereastra `User` și nu are de unde să știe ce e obiectul cu adevărat.

Și asta e **mai rău decât înainte**, nu mai bine. Înainte, fișierul rezultat era vizibil stricat: fără prefix, nu semăna cu ce aștepta cititorul, și crăpa imediat. Acum fișierul **arată** corect — are prefix, are forma bună, trece de `Guid.Parse` — dar prefixul minte. Un fișier care arată valid și conține date false e mai greu de depanat decât unul care crapă: eroarea nu mai apare la citire, apare peste două ecrane, sub forma „de ce nu mă mai pot loga ca admin?”.

Regula generală: **o valoare implicită plauzibilă e mai periculoasă decât o eroare**. Când codul nu are informația ca să răspundă corect, varianta bună e să se oprească, nu să pună ceva care seamănă a răspuns.

FIX: e T3, și e același de patru runde — ramificarea pe tip, în ambele sensuri. Metodele tale din `Student` / `Teacher` / `Admin` sunt deja scrise corect; se mută în `UserTextMapper` :

```
public string ToText(User item)
{
    if (item is Teacher t) { return TeacherToText(t); }
    if (item is Admin a)   { return AdminToText(a);   }
    if (item is Student s) { return StudentToText(s); }

    throw new ArgumentException("Tip de user necunoscut");
}
```

(Ordinea contează: `Admin` și `Teacher` înaintea lui `Student` doar dacă vreunul l-ar moșteni; aici sunt frați, deci ordinea e liberă — dar obiceiul de a pune tipul cel mai specific primul te scapă de bug-uri când ierarhia crește.)

B4 — (runda 4, a doua oară) Aplicația tot nu există. Acum și `try` -ul e comentat

`Program.cs:7-15` · `ViewLogIn.cs:8-69`

CUM E ACUM:

```

public static void Main()
{
    /*try
    {
        ViewLogIn logIn = new();
        logIn.Logger();
    }
    catch (ArgumentException text)
    {
        Console.WriteLine(text);
    }*/
}

```

`ViewLogIn.Logger` e în continuare comentat integral, cu `//User user = ...` comentat și înăuntru.

DE CE: mecanismul e scris în runda 4 și rămâne — `/* */` șterge codul pentru compilator, deci „build verde” nu mai înseamnă nimic despre partea comentată. Ce e nou e că, runda asta, buildul chiar a trecut. Și fix aici se vede prețul: **buildul verde te-a asigurat că merge un program care nu pornește**. Cele trei `FormatException` de la B1 și B2 sunt exact genul de lucru pe care un build nu-l poate prinde niciodată — apar la rulare, pe date reale.

Cât timp `Main` e gol, singurul lucru pe care îl verifici înainte de commit e că textul se compilează. Nu e suficient și nu a fost niciodată; acum e și singura ta plasă.

Blocajul e tot cel din runda 3: `UserService.GetByFirstAndLastName` n-a fost scrisă. Sunt patru linii, și sunt scrise mai jos pentru a treia oară.

FIX:

```

// UserService.cs
public User GetByFirstAndLastName(string firstName, string lastName)
{
    return repository.GetByFirstAndLastName(firstName, lastName);
}

// ViewLogIn.cs – scoate /* */, apoi:
User user = service.GetByFirstAndLastName(firstName, lastName);

// Program.cs – scoate /* */

```

B5 — Din patru `Int32.Parse` identice, ai șters trei

`ViewStudent.cs:69`

CUM E ACUM: meniul de cărți, singurul rămas:

```
tasta = Int32.Parse(Console.ReadLine());

if (!Int32.TryParse(Console.ReadLine(), out tasta))
```

Celelalte trei (`:36` , `:100` , `:125`) sunt curate. ✓

DE CE: e „repară linia, nu regula” în forma cea mai mică posibilă — patru locuri identice, adresa fiecăruia scrisă în review, trei bifate. Nu e o greșeală de înțelegere: ai înțeles perfect fix-ul, l-ai aplicat corect de trei ori.

E o greșeală de **verificare**. Ai reparat pe rând, din memorie, și te-ai oprit când ai simțit că ai terminat. Senzația aia nu e un criteriu; căutarea e. `Ctrl+Shift+F` pe `Int32.Parse` întoarce acum exact un rezultat — două secunde, și diferența dintre „cred că le-am făcut” și „știu că le-am făcut”.

Obiceiul pe care ți l-am cerut runda trecută era fix ăsta, și e singurul care închide tiparul care te urmărește de cinci runde: **după fiecare fix, caută tiparul în tot proiectul, înainte de a trece mai departe.**

FIX: șterge linia `:69` .

B6 — (a patra oară) Nimeni nu se poate loga. `UserTextMapper` tot nu ramifică pe tip

`Users/Mappers/UserTextMapper.cs:17` · `Users/Models/Student.cs:5` · `Teacher.cs:6` · `Admin.cs:5`

CUM E ACUM: partea bună, și e reală: `UserRepository` se construiește fără să crape, prima dată în trei runde. Prefixul pe care l-ai pus în `FromText` se potrivește, ca poziție, cu `STUDENT` , `ADMIN` , din fișier, deci cele 6 câmpuri se citesc.

Dar ce iese din `FromText` e mereu `new User(...)` . Deci în `ViewLogIn` , cele trei `as` dau toate `null` și se cade pe `throw new ArgumentException("Error on user type")` — exact ca în rundele 3 și 4.

DE CE: ai făcut jumătatea de format (prefixul) fără jumătatea de decizie (`if` -ul pe `cuv[0]`). Prefixul singur e o etichetă pe care n-o citește nimeni: el *poartă* informația de tip prin fișier, dar cineva trebuie să o și *folosească* ca să aleagă constructorul. Fără `if`, informația ajunge în program și e aruncată pe loc.

Iar cele șase metode corecte din `Student` / `Teacher` / `Admin` rămân necheamabile din motivul explicat la B3 runda 3: `ITextMapper<Student>` nu e `ITextMapper<User>`, iar `Repository<User>` cere al doilea. Nu se schimbă pentru că metodele dinăuntru sunt bune.

Ai acum toate piesele. Formatul e în fișier, metodele sunt scrise și verificate, `Repository<T>` e stabil, iar `Enrolment` tocmai ți-a arătat că știi să muți un tip întreg pe tiparul corect. Lipsește `if` -ul.

FIX:

```
public User FromText(string text)
{
    string[] cuv = text.Split(',');

    if (cuv[0] == "STUDENT") { return StudentFromText(cuv); }
    if (cuv[0] == "TEACHER") { return TeacherFromText(cuv); }
    if (cuv[0] == "ADMIN")    { return AdminFromText(cuv); }

    throw new ArgumentException("Tip de user necunoscut: " + cuv[0]);
}
```

Cele trei metode private se copiază din `Student.FromText`, `Teacher.FromText`, `Admin.FromText` — corpurile sunt deja bune — și apoi `ITextMapper<...>` iese din cele trei modele.

DoD, neschimbat de trei runde: pui un TEACHER în mijlocul lui `users.txt`, pornești, te loghezi cu el, salvezi, repornești — profesorul e tot profesor și are tot parola.

B7 — (runda 4, a doua oară) Garda pe email tot doar la `CreateStudent`

`Users/Services/UserService.cs:42` · `:55`

CUM E ACUM: neatins. `CreateTeacher` și `CreateAdmin` au în continuare `repository.FindById(teacher.Id)`, care e mereu `null` pentru că `Id` -ul s-a născut pe linia de deasupra. Singura schimbare din fișier runda asta sunt două linii goale adăugate la `:107-108`.

DE CE: același tipar ca la B5, dar cu adresele scrise explicit în review de două ori la rând. Mecanismul bug-ului e în runda 3; ce contează acum e că e a doua rundă în care fix-ul e copiat dintr-un loc în care l-ai scris deja corect. `CreateStudent` e chiar deasupra lui `CreateTeacher`, pe ecran, în același fișier.

FIX: copiază cele patru linii din `CreateStudent` în ambele metode, cu tipul potrivit.

● Importante

- **M1 — `EnrolmentService.GetEnrolmentIdByStudentId` întoarce id-uri de CURSURI.**
`EnrolmentService.cs:17-20` delegă spre `repository.GetCourseIdListByStudentId`, deși `EnrolmentRepository.cs:20` are metoda corectă, cu numele corect. Două metode cu nume diferite care întorc același lucru — una dintre ele minte. Nu explodează acum doar pentru că nimeni n-o cheamă; e o mină cu întârziere. Copy-paste între două delegări vecine.
- **M2 — (R4 M1, a doua oară) `EnrolmentRepository.FindById` ascunde metoda moștenită. :11**
• `warning CS0108`. `Repository<T>.FindById` (`Repository.cs:21`) face exact același lucru. Ai șters-o din `UserRepository` runda trecută; aici a rămas.
- **M3 — `EnrolmentService` construiește un repository pe care îl aruncă. :10 — `private readonly EnrolmentRepository repository = new();` inițializează câmpul (deci citește `enrolments.txt` de pe disc), iar apoi constructorul îl suprascrie cu cel primit. Fișierul se citește de două ori la fiecare `new EnrolmentService(...)`. Șterge `= new()`.**
- **M4 — (R4 M4, jumătate) verificarea de lungime la `Email` e tot înaintea lui `Trim`. `User.cs:97` vs `:102`.**
- **M5 — (R4 M5, NEATINS) `DeleteUser` nu verifică dacă userul există. `UserService.cs:109`.**
`DeleteEnrolment` ✓ verifică — ai scris fix-ul corect în celălalt serviciu, în același commit.
- **M6 — (R4 M6, NEATINS) nimic nu cheamă `Save()`, nici pentru useri, nici pentru înscrieri.**
`EnrolmentService` chiar și-a pierdut `Save()`-ul propriu la refactor, iar `repository.Save()` nu e chemat de nimeni: o înscriere nouă trăiește doar până la închiderea aplicației. (E T2, Observer — după T3.)
- **M7 — (R4 M8, NEATINS) modificarea unei cărți nu confirmă nimic. `ViewStudent.cs:218` — `response` atribuit și nefolosit.**
- **M8 — (R4 M9, NEATINS) login-ul de admin/profesor iese tăcut. `ViewLogIn.cs:39` / `:53`, în interiorul blocului comentat.**
- **M9 — (R4 M10, jumătate) `CourseService` se instanțiază singur și își citește fișierul.**
`ViewStudent.cs:15`. ✓ `BookService` și acum și `EnrolmentService` primesc repository-ul din afară — a rămas unul singur.

- **M10 — (R4 M11, NEATINS)** `GetEnrolmentIdByStudentId` din repository e cod mort. `EnrolmentRepository.cs:20`.
- **M11 —** `EnrolmentUpdateResponse(Guid id, ...)` — `id` cu literă mică, singurul din cele 16 DTO-uri.
- **M12 —** `UpdateEnrolment` scrie direct în entitate (`EnrolmentService.cs:64–65`), deși `EnrolmentMapper.ApplyUpdate` e scris exact pentru asta și nu e chemat de nimeni.

Cleanups

- **C1 — (a cincea oară)** tot nu există `.gitignore`. `bin/` și `obj/` untracked la mine, acum, după build.
 - **C2 — (a cincea oară)** `Console.WriteLine(aux);` rămas din depanare, `ViewStudent.cs:297`.
 - **C3 — (NEATINS)** `Data/students.txt` e mort. 0 referințe în cod.
 - **C4 — (NEATINS)** parole în clar în `Data/users.txt`.
 - **C5 —** 62 de linii comentate în `ViewLogIn.cs` + 10 noi în `Program.cs`. `git` ține tot.
 - **C6 —** 33 de warnings (44 → 35 → 33). Dintre ele 2 × `CS0108` (M2) și 12 × `CS8618`.
 - **C7 —** acolada clasei `Enrolment` e indentată greșit, `Enrolment.cs:21` — semn că fișierul n-a mai fost formatat (`Ctrl+K`, `Ctrl+D`).
 - **C8 —** două linii goale adăugate fără motiv, `UserService.cs:107–108`.
 - **C9 —** fișiere fără linie goală la final, acum inclusiv cele trei noi din `Enrolments/`.
-

Starea celor din runda 4

#	Runda 4	Acum	Cum am verificat
B1	nu compilează (1 vizibilă + 6 ascunse)	✅ ÎNCHIS — build verde, toți apelanții actualizați	<code>dotnet build</code>
B2	<code>Main</code> gol, <code>Logger</code> comentat	🔴 NEATINS (R5 B4) — acum și <code>try</code> e comentat	citit
B3	<code>TryParse</code> lipit sub <code>Parse</code> , x4	⚠️ trei sferturi — 3 șterse, <code>:69</code> rămas (R5 B5)	<code>grep</code>
B4	<code>users.txt</code> cu prefixe, <code>UserTextMapper</code> fără	⚠️ jumătate — fișierul se citește ✅, dar tot iese <code>User</code> (R5 B6)	rulat: <code>UserRepository</code> OK
B5	<code>Save()</code> rade parole și prefixe	🔴 înăutățit (R5 B3) — acum scrie <code>USER,</code> peste toate tipurile	rulat: <code>Save()</code> → 8 linii <code>USER,</code>
B6	garda pe email doar la <code>CreateStudent</code>	🔴 NEATINS (R5 B7)	citit: <code>FindById(teacher.Id)</code>
B7	<code>EnrolmentService</code> cu două liste paralele	✅ ÎNCHIS — o singură sursă, repository din afară	citit + rulat
M1	<code>EnrolmentRepository.FindById</code> ascunde	🔴 NEATINS (R5 M2)	<code>warning CS0108</code>
M2	<code>EnrolmentRepository</code> fără constructor	✅ ÎNCHIS	
M3	mapper-ele de <code>Enrolment</code> goale	✅ ÎNCHIS — ambele scrise (dar vezi R5 B2)	

#	Runda 4	Acum	Cum am verificat
M4	Email : lungimea înaintea lui Trim	🔴 NEATINS (R5 M4)	
M5	DeleteUser fără verificare	🔴 NEATINS (R5 M5)	
M6	Save() nechemat	🔴 înrautățit (R5 M6) — EnrolmentService și-a pierdut și Save() -ul	
M7	InscriereCurs ocolește serviciul	✅ ÎNCHIS	
M8– M11	confirmare lipsă, login tăcut, servicii duble, cod mort	⚠️ M10 jumătate (EnrolmentService ✅); restul NEATINSE	
C1– C9	.gitignore , aux , students.txt , parole, comentat, warnings	✅ C2 (User.cs); restul NEATINSE	
—	—	🔴 NOU: Books stricat (R5 B1)	rulat: ViewStudent crapă în ctor
—	—	🔴 NOU: EnrolmentTextMapper round-trip rupt (R5 B2)	rulat: FromText(ToText(e)) crapă

Din 7 🔴 : 2 închise, 2 pe jumătate, 3 deschise — plus 2 noi. Din 12 🟡 : 3 închise. Din 8 🟢 : 1.

Bilanțul pe restanțe e pozitiv a doua rundă la rând, și ai închis exact partea grea. Problema e alta și e nouă: **runda asta ai stricat cod care mergea**. Cele două 🔴 noi sunt amândouă „am schimbat un format și n-am pornit aplicația” — ceea ce, acum că buildul trece, e singurul lucru care te mai poate opri.

Before / After pentru cele

#	Acum	Cum ar trebui
B1	<pre>return "BOOK," + item.Id + ... new Book(Guid.Parse(cuv[1]), ..., cuv[3], cuv[4])</pre>	fără prefix: <pre>item.Id + ... new Book(Guid.Parse(cuv[0]), ..., cuv[2], cuv[3])</pre>
B2	<pre>ToText : 4 câmpuri, fără prefix, item.CreatedAt FromText : cuv[1] ... cuv[4]</pre>	ambele pe 4 câmpuri, fără prefix, <pre>CreatedAt.ToString("yyyy-MM-dd") , FromText de la cuv[0]</pre>
B3	<pre>ToText(User item) → "USER," + ... (5 câmpuri)</pre>	<pre>if (item is Teacher t) return TeacherToText(t); ... cu toate câmpurile tipului</pre>
B4	<pre>public static void Main() { /*... */ }</pre>	<pre>try { new ViewLogIn().Logger(); } catch ...</pre>
B5	<pre>tasta = Int32.Parse(Console.ReadLine()); if (!Int32.TryParse(...))</pre>	doar <pre>if (!Int32.TryParse(...)) { InputGresit(); continue; }</pre>
B6	<pre>FromText → new User(Guid.Parse(cuv[1]), ...)</pre>	<pre>if (cuv[0] == "STUDENT") return StudentFromText(cuv); ...</pre>
B7	<pre>Teacher teacher = ToTeacher(request); if (repository.FindById(teacher.Id) != null)</pre>	<pre>if (repository.GetByEmail(request.Email) != null) { throw ... } apoi construiești</pre>

Întrebări de verificare

1. **B1.** Ai pus prefix "BOOK," în `BookTextMapper` și prefix "USER," în `UserTextMapper`. Unul dintre cele două prefixe rezolvă o problemă reală, celălalt doar strică un fișier. Care e care, și **ce întrebare** trebuie să-și pună un fișier ca să aibă nevoie de prefix?

2. **B2.** `EnrolmentTextMapper.ToText` și `FromText` sunt la patru rânduri distanță, în același fișier, scrise în aceeași ședință — și nu se potrivesc. Runda trecută ai închis exact același bug la `Teacher` și `Admin`. Ce ai făcut atunci și n-ai făcut acum? (Răspunsul e o acțiune, nu o intenție.)
 3. **B3.** Runda 4, `Save()` scria linii **fără** prefix și aplicația crăpa la următoarea pornire. Runda 5, scrie linii **cu** prefixul `USER,` și aplicația pornește. Argumentează de ce a doua variantă e mai periculoasă decât prima.
 4. **B5.** Ai șters trei din patru `Int32.Parse`. Nu te întreb care a rămas — ți-am scris adresa. Te întreb: ce comandă sau combinație de taste ți-ar fi arătat, în două secunde, că mai e unul? Scrie-o exact.
 5. **Proces.** Buildul a trecut runda asta, prima dată. Și tot runda asta ai livrat trei `FormatException` la pornire, în trei repository-uri din trei. Ce verificare, în plus față de `dotnet build`, trebuie să intre în rutina ta înainte de fiecare commit — și de ce buildul nu putea, prin construcție, să prindă nimic din cele trei?
-

Ce urmează

Ordinea de data asta e scurtă, pentru că ești foarte aproape.

1. **Repară cele două formate** — B1 (scoate prefixul din `Books`) și B2 (potrivește cele două jumătăți din `EnrolmentTextMapper` + `ToString("yyyy-MM-dd")`). Zece minute.
2. **Pune `Main` și `Logger` înapoi** — B4, plus `UserService.GetByFirstAndLastName`. Patru linii, a treia oară când ți le scriu.
3. **Pornește aplicația și fă un tur complet:** login → cărți → cursuri → statistici → ieșire. Dacă crapă, ai găsit ce nu prinde buildul.
4. **Șterge `Int32.Parse` de la `:69`** — B5.
5. **T3, Factory Method** — B6 + B3. `if`-ul pe `cuv[0]` în `FromText`, `is` pe tip în `ToText`, cele șase metode mutate din modele. **Asta e tot ce a mai rămas din blocajul de patru runde.** DoD: TEACHER în mijlocul fișierului → login → `Save()` → repornire → profesorul e tot profesor, cu parolă.
6. **B7** — două metode, copiate din `CreateStudent`.

Un commit per pas. **Build verde ȘI un tur prin aplicație înainte de fiecare commit** — regula veche își schimbă forma, pentru că de runda asta buildul singur nu mai e de ajuns.

Runda 4 — 2026-09-15 — sha `aba4e01`

Verificat commit-ul „code review fixes”. Buildul a picat a șasea oară — **1 eroare vizibilă, 6 ascunse în spatele ei**, pentru că `Enrolment` nu implementa `IEntity` și legarea tipului de bază oprea verificarea corpurilor. Prima rundă cu bilanț pozitiv pe restanțe (3  + 5  închise), dar aplicația a fost **golită** ca să treacă buildul.

#	Constatare	Stare azi
B1	nu compilează: <code>Enrolment</code> fără <code>IEntity</code> + 6 erori ascunde (apelanți neactualizați după mutare)	✅ ÎNCHIS — build verde, toți apelanții updatați
B2	aplicația nu mai există: <code>Main</code> gol, <code>Logger</code> comentat integral	🔴 NEATINS (R5 B4)
B3	<code>TryParse</code> lipit sub <code>Int32.Parse</code> , x4 → meniul stricat și pentru input corect	⚠️ trei sferturi — <code>:69</code> rămas (R5 B5)
B4	<code>users.txt</code> a trecut la prefixe, <code>UserTextMapper</code> nu → contradicție garantată	⚠️ jumătate — fișierul se citește ✅, dar tot iese <code>User</code> (R5 B6)
B5	<code>Save()</code> rade salarii/parole și șterge prefixele	🔴 Înrăutățit (R5 B3) — scrie <code>USER</code> , peste toate tipurile
B6	garda pe email doar la <code>CreateStudent</code>	🔴 NEATINS (R5 B7)
B7	<code>EnrolmentService</code> cu două liste paralele; <code>repository</code> nefolosit	✅ ÎNCHIS
M1	<code>EnrolmentRepository.FindById</code> ascunde metoda moștenită	🔴 NEATINS (R5 M2)
M2	<code>EnrolmentRepository</code> fără constructor <code>base(...)</code>	✅ ÎNCHIS
M3	<code>EnrolmentMapper</code> / <code>EnrolmentTextMapper</code> clase goale	✅ ÎNCHIS (dar round-trip rupt — R5 B2)
M4	<code>Email</code> : lungimea verificată înaintea lui <code>Trim</code>	🔴 NEATINS
M5	<code>DeleteUser</code> nu verifică dacă userul există	🔴 NEATINS
M6	nimic nu cheamă <code>Save()</code> pentru useri	🔴 Înrăutățit — și <code>EnrolmentService</code> și-a pierdut <code>Save()</code>

#	Constatare	Stare azi
M7	<code>InscriereCurs</code> ocolește serviciul	✅ ÎNCHIS
M8–M12	confirmare lipsă la <code>UpdateBook</code> , login tăcut, servicii auto-instanțiate, cod mort, naming DTO	⚠️ M10 jumătate; restul NEATINSE
C1–C8	<code>.gitignore</code> , cod comentat, <code>students.txt</code> , <code>aux</code> , parole, warnings, linie goală, newline la final	✅ C2 (<code>User.cs</code>); restul NEATINSE

Mecanismele explicate atunci și încă valabile: **o eroare de legare a unui tip suprimă erorile din tot codul care îl folosește** (B1) · `/* */` scoate codul din build, deci „build verde” devine un semnal fals (B2) · `Console.ReadLine()` **consumă** o linie la fiecare apel, deci două apeluri cer două linii (B3) · **tipul static al parametrului decide ce câmpuri sunt vizibile**, de-aia `ToText(User)` nu poate scrie un salariu (B5) · o constatare are un *unde* și o *regulă*; adresa e doar punctul din care ți-am arătat regula (B6).

Runda 3 — 2026-09-09 — sha `70360d6`

Verificat commit-ul „more user fixes”. Buildul a picat cu 4 erori. Partea de fond s-a mișcat serios (B4 din R2 închis pe bune), dar `Student` / `Teacher` / `Admin` au primit `ITextMapper<T>` — șase metode pe care nimeni nu le putea chema.

#	Constatare	Stare azi
B1	nu compilează: <code>ViewLogIn:10</code> , <code>:21</code> , <code>Teacher.cs:87</code> — apelanți neactualizați după mutare	🔴 redeschis (R4 B1), același tipar
B2	aplicația nu pornește; după reparare, nici studenții nu se pot loga	🔴 NEATINS (R4 B4)
B3	<code>ITextMapper<Student></code> nu e <code>ITextMapper<User></code> → cele 6 metode noi, necheamabile; <code>Save()</code> rade parole	🔴 NEATINS (R4 B4, B5)
B4	fiecare <code>FromText</code> contrazice <code>ToText</code> -ul de deasupra (prefix + câmpuri lipsă)	✅ ÎNCHIS — round-trip verde pe toate trei
B5	<code>Admin</code> : <code>salary</code> înaintea lui <code>age</code> → admin de 10000 de ani cu salariul 48	✅ ÎNCHIS
B6	<code>if (FindById(student.Id) != null)</code> — mereu falsă, <code>Id</code> proaspăt generat	⚠️ o treime — <code>CreateStudent</code> ✅ , <code>CreateTeacher</code> / <code>CreateAdmin</code> 🔴 (R4 B6)
B7	<code>Int32.Parse</code> pe tastă, 4 locuri	🔴 Înrăutățit (R4 B3) — <code>TryParse</code> lipit sub <code>Parse</code>
B8	<code>Console.WriteLine</code> în loc de <code>throw</code> la <code>Salary</code> / <code>Password</code>	✅ ÎNCHIS , toate patru
M1	<code>UserRepository.FindById</code> ascunde metoda moștenită	✅ ÎNCHIS — refăcut în <code>EnrolmentRepository</code> (R4 M1)
M2	<code>Email</code> nu se curăță de spații	⚠️ jumătate — <code>email = text</code> ✅ , lungimea tot pre- <code>Trim</code> (R4 M4)

#	Constatare	Stare azi
M3	<code>UserMapper</code> nu e <code>static</code>	✅ ÎNCHIS
M4	<code>ToStudentUpdateResponse(User)</code> față de frații ei	✅ ÎNCHIS
M5	nimic nu cheamă <code>Save()</code> pentru useri	❌ NEATINS (R4 M6)
M6	<code>DeleteUser</code> nu verifică dacă userul există	❌ NEATINS (R4 M5)
M7	<code>StudentCreateRequest</code> în alt namespace	✅ ÎNCHIS
M8–M12	cod mort, <code>InscriereCurs</code> , confirmare lipsă, login tăcut, servicii duble	❌ NEATINSE (R4 M7–M11)
C1–C9	cod comentat, <code>.gitignore</code> , <code>students.txt</code> , <code>aux</code> , parole, warnings, <code>using</code> -uri, newline, naming	✅ C7; C1 ❌ înrăutățit (+62 de linii); restul NEATINSE

Fix-urile complete, cu cod, pentru fiecare dintre cele de mai sus sunt în runda 4 (pentru cele încă deschise) — iar mecanismele explicate atunci rămân valabile: invarianța generică la B3, round-trip-ul la B4, „`int` e `int`” la B5, identitate vs. date la B6.

Runda 2 — 2026-09-09 — sha `3c1877d`

Verificat pasul 3 din Faza 1 (`User` → T0.5). Buildul a picat cu 16 erori (6 vizibile, 10 ascunse în spatele lor). Pașii 1 și 2 — `Course` și `Enrolment` — sări.

#	Constatare	Stare azi
B1	nu compilează: metode duplicate în <code>UserMapper</code> + DTO-uri șterse fără apelanți	🔴 redeschis (R3 B1)
B2	<code>UserTextMapper.FromText</code> face <code>Guid.Parse</code> pe prefixul de tip → aplicația nu pornește	🔴 NEATINS (R3 B2)
B3	<code>FromText</code> întoarce mereu <code>User</code> → adminul devine student, <code>Save()</code> rade parola și salariul	🔴 NEATINS (R3 B2, B3)
B4	<code>CreateUser</code> : <code>if (request != null)</code> mereu adevărat → orice cerere produce un <code>Student</code>	✅ ÎNCHIS
B5	<code>Salary / Password</code> în <code>Teacher</code> și <code>Admin</code> : <code>Console.WriteLine</code> în loc de <code>throw</code>	🔴 NEATINS (R3 B8)
B6	<code>Int32.Parse</code> pe tastă în <code>ViewStudent</code>	🔴 NEATINS (R3 B7)
M1	<code>UserRepository.FindById</code> ascunde metoda moștenită (<code>CS0108</code>)	🔴 NEATINS
M2	<code>Email</code> nu se face <code>Trim</code> , deși <code>FirstName / LastName</code> da	🔴 NEATINS
M3	<code>UserMapper</code> nu e <code>static</code> ; <code>ToTeacher</code> întoarce <code>User</code>	⚠️ jumătate

#	Constatare	Stare azi
M4	ordinea argumentelor: <code>Teacher</code> greșit, <code>Admin</code> — capcană anunțată	⚠️ <code>Teacher</code> ✅ / <code>Admin</code> ❌ (R3 B5)
M5	<code>StudentCreateRequest</code> în alt namespace decât frații lui	❌ NEATINS
M6–M10	cod mort, <code>InscriereCurs</code> ocolește serviciul, modificarea cărții nu confirmă, login admin/profesor tăcut, servicii instanțiate de mai multe ori	❌ NEATINS
C1–C7	cod comentat, <code>.gitignore</code> , <code>students.txt</code> mort, <code>Console.WriteLine(aux)</code> , parole în clar, warnings, <code>using</code> -uri	⚠️ C1 parțial; restul NEATINSE

Runda 1 — 2026-09-08 — sha `79abec3`

Verificat T0.0–T0.4 din `TASKS.md`. Buildul a trecut. **5/5 task-uri închise pe fond.**

#	Constatare	Stare azi
T0.0–T0.4	nu compilează · „Vezi cursurile” crapă · modificarea orfanizează cartea · <code>createdAt</code> ignorat · <code>CreateUser</code> face mereu un <code>User</code> gol	✅ toate 5 închise
B1	<code>Contains</code> la căutarea după nume → Enter pe gol înseamnă „primul din listă”	✅ ÎNCHIS
B2	<code>null</code> strecurat în lista de cursuri	✅ ÎNCHIS
B3	profesorul salvat nu mai poate fi citit (<code>ToText</code> ≠ constructorul din text)	❌ reapărut de 3 ori în modele (R3 B4)
B4	<code>Int32.Parse</code> pe tastă	❌ NEATINS de 2 runde (R3 B7)
M1	<code>Create</code> / <code>Update</code> din modele = cod mort	✅ șterse
M2	<code>BookUpdateRequest</code> cere o dată ignorată	✅ ÎNCHIS de T0.5
M3	<code>GetEnrolmentIdByStudentId</code> cod mort	❌ NEATINS
M4	modificarea cărții nu confirmă nimic	❌ NEATINS
M5	<code>User newUser = new();</code> inutil	✅ ÎNCHIS
M6	<code>InscriereCurs</code> ocolește serviciul	❌ NEATINS

#	Constatare	Stare azi
M7	login admin/profesor iese tăcut	🔴 NEATINS
M8	serviciile se instanțiază de mai multe ori	⚠️ <code>BookService</code> ✅, restul 🔴
C1-C7	<code>Console.WriteLine(aux)</code> · <code>.gitignore</code> · <code>students.txt</code> · parole în clar · warnings · <code>override</code> gol · <code>using</code> -uri	✅ C6; restul 🔴



MyCodeSchool

Material didactic MyCodeSchool

mycodeschool.ro